

PROPOSTA DE TRABALHO DE GRADUAÇÃO

Otimização de Latência de Cauda (P99) na Entrega de Modelos de Aprendizado de Máquina para Dados Tabulares

Aluno: David Ubaldo Mesquita Londres

Orientador: Cleber Zanchettin

Área: Inteligência Artificial / Aprendizado de Máquina

Curso: Ciência da Computação

Sugestões para Avaliação: Flavio Santos / Adriano Lorena

Cleber Zanchettin
Orientador — Assinatura Digital

Recife, Abril de 2026

Resumo

Em ambientes de produção, a entrega eficiente de inferências de modelos de aprendizado de máquina é tão crítica quanto a acurácia do modelo. Fenômenos como *cold starts* e *jitter* de sistema degradam a latência de cauda (P99), tornando a mediana (P50) uma métrica insuficiente para avaliar a confiabilidade operacional. Este trabalho propõe uma análise quantitativa comparativa da latência P99 ao servir um modelo XGBoost, treinado sobre um dataset tabular de grande escala e exportado em formato ONNX, a partir de dois pipelines: um baseado em FastAPI e outro em BentoML com *adaptive batching*. Um protocolo de benchmarking com fase de aquecimento e taxa de requisições constante será utilizado para gerar distribuições de latência estatisticamente válidas. O entregável principal é a análise empírica das distribuições de latência, identificando sob quais condições de carga cada abordagem apresenta vantagens em termos de confiabilidade P99.

Palavras-chave: Inferência de ML; latência de cauda P99; dados tabulares; XGBoost; ONNX; BentoML; FastAPI; benchmarking.

1 Introdução

A otimização de inferência de modelos de aprendizado de máquina representa uma interseção entre os algoritmos de ciência de dados e as exigências práticas de engenharia de sistemas.¹ Enquanto a comunidade acadêmica dedica grande atenção ao treinamento de modelos, a entrega eficiente de predições em produção permanece subexplorada, especialmente para *ensembles* de árvores de decisão.²

Existe, contudo, um abismo entre treinar um modelo em um *Jupyter Notebook* e colocá-lo em produção com latência previsível e escalabilidade. Em domínios que servem dados tabulares de forma síncrona, como detecção de fraude em tempo real, onde cada transação bancária exige uma decisão em milissegundos para bloquear ou aprovar a operação; escoragem de crédito, onde a experiência de aprovação instantânea depende de um modelo tabular servido via API; e *digital advertising*, onde lances de *Real-Time Bidding* (RTB) impõem orçamentos de latência inferiores a 10 ms, a latência de cauda P99 não é uma métrica acadêmica: é a diferença entre capturar ou perder uma transação.

Modelos de *gradient boosting*, como o XGBoost,³ dominam essas aplicações de dados tabulares, superando redes neurais em benchmarks tabulares.⁴ Entretanto, servir esses modelos com garantias de latência é um desafio técnico distinto: a travessia condicional de centenas de árvores binárias causa padrões irregulares de acesso à memória.¹

Em arquiteturas modernas de microsserviços, sistemas de ML são frequentemente implantados em contêineres orquestrados via Docker ou Kubernetes.⁵ Nesses ambientes, *cold starts* (inicialização de novas instâncias) e *jitter* de sistema (variações de latência por pausas de *garbage collector*, trocas de contexto de CPU e *cache misses*)⁶ tornam a mediana de latência (P50) uma métrica inadequada para avaliar a confiabilidade do sistema. A latência de cauda, representada pelo percentil P99, captura as anomalias severas que afetam a experiência dos usuários.⁷

Este trabalho propõe comparar dois pipelines de entrega de modelos tabulares, FastAPI e BentoML, focando na análise das distribuições de latência P99 sob diferentes níveis de carga, com o objetivo de fornecer evidência empírica que auxilie engenheiros a selecionar a arquitetura de *serving* mais adequada para sistemas onde cada milissegundo de cauda tem impacto direto em receita e experiência do usuário.

2 Objetivos

2.1 Objetivo Geral

Produzir uma análise quantitativa comparativa da latência de cauda (P99) ao servir um modelo tabular otimizado através de dois pipelines de entrega distintos: um framework web generalista (FastAPI) e uma plataforma especializada de ML (BentoML).

2.2 Objetivos Específicos

1. Treinar um modelo XGBoost sobre um dataset tabular de classificação binária e exportá-lo no formato ONNX;
2. Construir dois pipelines de inferência containerizados: (a) FastAPI com ONNX Runtime e (b) BentoML com *adaptive batching*;
3. Executar um protocolo de benchmarking com fase de aquecimento e taxa de requisições constante;
4. Registrar distribuições de latência (P50, P95, P99) e *throughput* para ambos os pipelines sob diferentes níveis de concorrência;
5. Analisar estatisticamente os resultados, identificando as condições sob as quais cada abordagem apresenta vantagens na compressão da cauda P99.

3 Metodologia

3.1 Preparação de Dados e Modelo

Um dataset tabular de classificação binária de grande escala será selecionado, pré-processado e utilizado para treinar um modelo XGBoost. O modelo será exportado para ONNX via `onnxmltools`.⁸

3.2 Pipelines de Inferência

Dois pipelines containerizados via Docker serão construídos com recursos de CPU e memória controlados:

- **Pipeline A — FastAPI:** Aplicação assíncrona servindo o modelo via ONNX Runtime, representando a abordagem generalista;
- **Pipeline B — BentoML:** O mesmo modelo empacotado com *adaptive batching*, agrupando requisições em lotes antes da inferência.

Em ambos os pipelines, os hiperparâmetros de concorrência do ONNX Runtime (`intra_op_num_threads` e `inter_op_num_threads`) serão fixados de acordo com os limites de CPU alocados ao contêiner Docker, garantindo reprodutibilidade e isolamento dos efeitos de paralelismo do *runtime* sobre a latência medida.

3.3 Protocolo de Benchmarking

O protocolo seguirá práticas estabelecidas de benchmarking de inferência.^{7,9}

1. **Aquecimento:** 50–100 requisições disparadas antes da medição para inicialização de compiladores JIT e preenchimento de caches;
2. **Medição sustentada:** um gerador de carga de modelo aberto (Vegeta)¹⁰ disparará requisições a taxa constante por 60 segundos. Diferentemente de ferramentas de modelo fechado, que desaceleram o envio quando o servidor engasga (*Coordinated Omission*)⁷, o Vegeta mantém a taxa de disparo independentemente do tempo de resposta, registrando a latência real experimentada pelo cliente;
3. **Variação de concorrência:** os testes serão executados com 1, 10, 50 e 100 clientes simultâneos.

3.4 Análise de Resultados

Para cada configuração, serão registradas as distribuições de latência (P50, P95, P99) e *throughput* (RPS). Os resultados serão analisados estatisticamente, com foco na razão P99/P50 como indicador de *jitter* e na identificação do ponto de inflexão de concorrência entre as duas abordagens.

4 Cronograma

Tabela 1: Cronograma de atividades do projeto.

Atividade	Mês 1	Mês 2	Mês 3	Mês 4
Revisão bibliográfica e treinamento do modelo	•			
Exportação ONNX e construção dos pipelines	•	•		
Implementação do protocolo de benchmarking		•		
Execução dos experimentos e coleta de dados			•	
Análise estatística e redação da monografia			•	•
Revisão final e apresentação				•

Referências

- [1] Supun Nakandala, Yuhao Yu e Arun Kumar. “A Comparison of End-to-End Decision Forest Inference Pipelines”. Em: *Proceedings of the VLDB Endowment*. Disponível em: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12406228/>. 2020.
- [2] Zeynab Asadi e Sandeep Tata. “A Comparison of Decision Forest Inference Platforms from A Database Perspective”. Em: *arXiv preprint arXiv:2302.04430* (2024). Disponível em: <https://arxiv.org/pdf/2302.04430>.
- [3] Tianqi Chen e Carlos Guestrin. “XGBoost: A Scalable Tree Boosting System”. Em: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794.
- [4] Léo Grinsztajn, Edouard Oyallon e Gaël Varoquaux. “Why do tree-based models still outperform deep learning on typical tabular data?” Em: *Advances in Neural Information Processing Systems* 35 (2022).
- [5] Matei Zaharia et al. “Scalable and Secure AI Inference in Healthcare: A Comparative Benchmarking of FastAPI and Triton Inference Server on Kubernetes”. Em: *arXiv preprint arXiv:2602.00053* (2024). Disponível em: <https://arxiv.org/abs/2602.00053>.
- [6] Jeffrey Dean e Luiz André Barroso. “The Tail at Scale”. Em: *Communications of the ACM*. Vol. 56. 2. 2013, pp. 74–80.
- [7] Gil Tene. *How NOT to Measure Latency*. P99 CONF. Disponível em: <https://www.youtube.com/watch?v=m8nq3cbMgY4>. 2015.
- [8] Microsoft. *Quantize ONNX models — ONNX Runtime Documentation*. Disponível em: <https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html>. 2024.
- [9] Vijay Janapa Reddi et al. “MLPerf Inference Benchmark”. Em: *Proceedings of the ACM/IEEE International Symposium on Computer Architecture*. 2020.
- [10] Tomás Senart. *Vegeta: HTTP load testing tool and library*. Disponível em: <https://github.com/tsenart/vegeta>. 2024.